luatexja-ruby パッケージ

LuaTEX-ja プロジェクトチーム

2026-08-30 v0.71（2026 年 10 月 17 日）

概要

luatexja-ruby パッケージは，LuaTEX-ja の機能を利用してルビの組版処理を行う追加パッケージである．LuaTEX, LuaTEX-ja の内部処理に割り込むことにより，熟語ルビ中の行分割や，行頭形・行中形・行末形の自動検出，また進入許容量の自動設定などを可能とした．

v0.3 より前とは親文字の高さの扱いが変わっている（`baseheight` キーを参照せよ）ほか，rubypreintrusion, rubypostintrusion パラメータの初期値が変わっているので注意すること．

目次

1 利用方法 2

1.1 用語 2

1.2 命令 2

1.3 グループの指定 7

2 注意点 8

3 実装について 10

3.1 進入量の計算 10

3.2 ノードの扱い 11

4 いくつかの例 13

5 『日本語組版処理の要件』20120403 の例 16

1 利用方法（りようほうほう）

パッケージ読み込みは，`\usepackage{luatexja-ruby}` で良く，必要ならば自動的に LuaTEX-ja 本体を読み込む．plain LuaTEX でのロードはまだサポートしておらず，LATEX 2_ε のみサポートしている．

1.1 用語（ようご）

「進入 (intrusion)」「突出 (protrusion)」という用語は，pxrubrica パッケージでの用法に準ずる．

進入あり：と暁（あかつき）の
進入なし：と 暁（あかつき） の
突出あり：聴衆（ちょうしゅう）
突出なし：聴　衆（ちょうしゅう）

なお，本パッケージでは親文字と直前・直後の文字の間に 0 でない和文処理グルー[*1]がくることも考慮しているため，「**前後の文字への**進入（許容）量」と「進入（許容）量」とは異なる可能性がある．この文書では次のように称する：

文字進入量 前後の文字へルビ文字が実際にかかる長さ．常に下線を引くことにする．
進入量 前後の文字，およびそれとルビの間の和文処理グルーにかかる長さ

多くの場合，和文処理グルーは 0 以上の長さのため，進入量は文字進入量以上である．
例えば次の例では，直前の文字「来」への前文字進入量は 0 であるが，前進入量は（和欧文間空白にかかる分まで含めるので）正である．

本来|ほげほげふがふが| foohoge においては……
本来 foohoge においては……

また，「親文字全角幅」「親文字全角高さ」という用語は，それぞれ親文字の和文フォントにおける `\zw`, `\zh` 命令で取得できる値を表す．多くの場合は両者は同じ値であるが，いつでもそうであるという保証はない．「ルビ全角幅」「ルビ全角高さ」についても同様である．

1.2 命令（めいれい）

■**`\ltjruby`** ルビ出力用命令の本体．`\ruby` という別名を定義している．

`\ltjruby[`⟨*option*⟩`]{親|文|字}{おや|も|じ}`

のように親文字→ルビの順序で指定する．第 2・第 3 引数内の「|」はグループの区切りを表す．詳細は 1.3 小節を参照．

⟨*option*⟩ には以下の内容を key-value リストで指定可能である．⟨*real*⟩ は 10 進の実数値を表す．⟨*bool*⟩ は真偽値 `true`（真）か `false`（偽）であり，値を省略したときには `true` の意味になる．

`pre=`⟨*real*⟩ 前文字進入許容量をルビ全角単位で指定．負の長さを指定した場合は，ルビの状況や直前の文字に応じた自動指定を意味する．既定値は負（つまり，自動指定）．
`post=`⟨*real*⟩ 同様に，後文字進入許容量を指定する．既定値は負（自動指定）．

[*1] JFM で指定されたグルーや，標準の和文間空白 (kanjiskip)，標準の和欧文間空白 (xkanjiskip)．

mode 進入処理のモードを表す bit vector．下位 2 bit は，pre や post が負である場合にしか効力を発揮しない．既定値は $(00001)_2 = 1$.

bit 0 前後の文字への進入を無効にするならば 0，有効にするならば 1.

bit 1 前進入許容量 B と後進入許容量 A が異なった場合，そのまま処理する場合は 0，小さい方に揃えるならば 1.

bit 2–4 ルビ文字の突出量から実際の前・後進入量の計算方法を指定する．詳しい計算方法については 3.1 小節を参照.

intrude_jfmgk=⟨*bool*⟩ 進入量算出の際に，前後の JFM グルーの自然長を考慮するか否か．既定値は真.

intrude_kanjiskip=⟨*bool*⟩ 進入量算出の際に，ルビ前後に挿入される標準の和文間空白 (kanjiskip) の自然長を考慮するか否か．既定値は真.

intrude_xkanjiskip=⟨*bool*⟩ 進入量算出の際に，ルビ前後に挿入される標準の和欧文間空白 (xkanjiskip) の自然長を考慮するか否か．既定値は真.

stretchruby={⟨*left*⟩**}{**⟨*middle*⟩**}{**⟨*right*⟩**}** 親文字の合計長がルビ文字の合計長より長い時に，ルビ文字の前・間・後に入れる空白の割合であり，それぞれ 0–7 の自然数で指定する．既定値は {1}{2}{1} である．⟨*left*⟩ はルビ文字の先頭までの空き量，⟨*middle*⟩ はルビ文字間の空き量，⟨*right*⟩ はルビ文字の末尾からの空き量（の比）を表す．以下が例である．



```
\ltjruby[stretchruby=123,maxmargin=2]%
  {○○○○}{◆◆}
```

stretch={⟨*left*⟩**}{**⟨*middle*⟩**}{**⟨*right*⟩**}** 行中形でルビ文字の方が長い場合，親文字の前・中・後に入れる空白の割合．既定値は {1}{2}{1} である．それ以外の代表的な値としては，例えば次のようなものがある.

親文字均等割禁止 {1}{0}{1} など ⟨*middle*⟩ を 0 にした値

前突出禁止 {0}{1}{1}

後突出禁止 {1}{1}{0}

stretchbol={⟨*left*⟩**}{**⟨*middle*⟩**}{**⟨*right*⟩**}** 行頭形に対する stretch と同様の指定．既定値は {0}{1}{1} である.

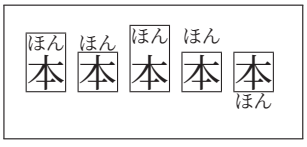
stretcheol={⟨*left*⟩**}{**⟨*middle*⟩**}{**⟨*right*⟩**}** 行末形に対する stretch と同様の指定．既定値は {1}{1}{0} である.

maxmargin=⟨*real*⟩ 親文字の方がルビより長い時に，ルビの先頭と親文字の先頭，及びルビ末尾と親文字の末尾の間に許される最大の空白量．**親文字全角幅単位**で指定し，既定値は 0.5.

size=⟨*real*⟩ ルビ文字の親文字に対する大きさ．既定値は 0.5.

intergap=⟨*real*⟩ ルビ文字と親文字との空きを親文字全角高さ単位で指定．既定値は 0.

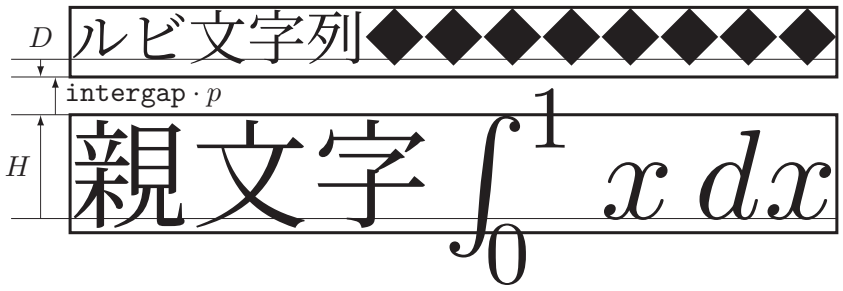
rubysmash=⟨*bool*⟩ ルビの高さを 0 にするか．既定値は偽．次が例である.



```
\ \vrule width 0pt height 2\zw depth 1\zw
\frame{\ltjruby[rubysmash=false]{本}{ほん}}\
\frame{\ltjruby[rubysmash=true]{本}{ほん}}\
\frame{\ltjruby[rubysmash=false,intergap=0.2]{本}{ほん}}\
\frame{\ltjruby[rubysmash=true,intergap=0.2]{本}{ほん}}\
\frame{\ltjruby[rubysmash=false,intergap=-1.5]{本}{ほん}}
```

ybaseheight=⟨*real*⟩ 非負の値が指定された場合，**縦組以外での**親文字の高さを親文字全角高さの ⟨*real*⟩ 倍と強制的に固定する．負の値が指定された場合は「固定しない」（すなわち，v0.3 より前の挙動と同じになる）．既定値は 0.88.

tbaseheight=⟨*real*⟩ ybaseheight と同様だが，こちらは**縦組での**親文字の高さを指定する．既定



ここで p は親文字における全角高さ，r はルビ文字における全角高さであり，

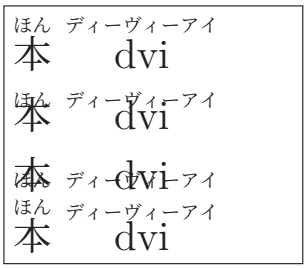
$$H = \begin{cases} \texttt{[yt]baseheight} \cdot p & \text{if } \texttt{[yt]baseheight} \geq 0, \\ \text{(自然高さ)} & \text{if } \texttt{[yt]baseheight} < 0, \end{cases}$$

$$D = \begin{cases} \texttt{[yt]rubydepth} \cdot r & \text{if } \texttt{[yt]rubydepth} \geq 0, \\ \text{(自然深さ)} & \text{if } \texttt{[yt]rubydepth} < 0. \end{cases}$$

図 1　ルビの垂直配置

値は 0.5.

baseheight=⟨*real*⟩ ybaseheight，tbaseheight を同時に指定したことと同義．次の例や図 1 を参照.

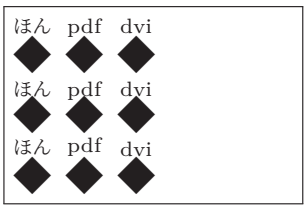



```
\noindent
\ltjruby[baseheight=0.88]{本}{ほん}\
\ltjruby[baseheight=0.88]{dvi}{ディーヴィーアイ}\\
\ltjruby[baseheight=0.5]{本}{ほん}\
\ltjruby[baseheight=0.5]{dvi}{ディーヴィーアイ}\\
\ltjruby[baseheight=0]{本}{ほん}\
\ltjruby[baseheight=0]{dvi}{ディーヴィーアイ}\\
\ltjruby[baseheight=-1]{本}{ほん}\
\ltjruby[baseheight=-1]{dvi}{ディーヴィーアイ}
```

yrubydepth=⟨*real*⟩ 非負の値が指定された場合，**縦組以外での**ルビの深さをルビ全角の ⟨*real*⟩ 倍と強制的に固定する．負の値が指定された場合は「固定しない」（すなわち，v0.52 以前の挙動と同じになる）．既定値は 0.12.

trubydepth=⟨*real*⟩ yrubydepth と同様だが，こちらは**縦組での**ルビの深さを指定する．既定値は 0.5.

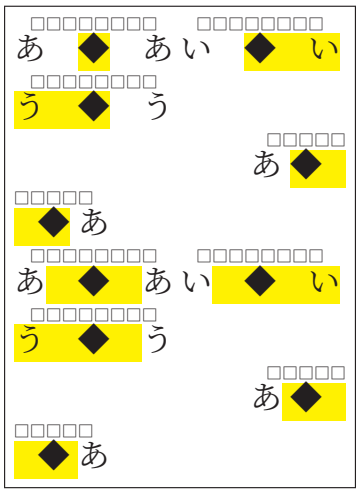
rubydepth=⟨*real*⟩ yrubydepth，trubydepth を同時に指定したことと同義．次の例や図 1 を参照.




```
\noindent
\ltjruby[rubydepth=0.12]{◆}{ほん}\
\ltjruby[rubydepth=0.12]{◆}{pdf}\
\ltjruby[rubydepth=0.12]{◆}{dvi}\\
\ltjruby[rubydepth=0]{◆}{ほん}\
\ltjruby[rubydepth=0]{◆}{pdf}\
\ltjruby[rubydepth=0]{◆}{dvi}\\
\ltjruby[rubydepth=-1]{◆}{ほん}\
\ltjruby[rubydepth=-1]{◆}{pdf}\
\ltjruby[rubydepth=-1]{◆}{dvi}
```

kenten=⟨*command*⟩ 各文字につく圏点の出力命令を指定する．既定値は「\textbullet」である.

fontcmd=⟨*command*⟩ ルビ用のフォント切り替え命令を指定する．このキーの内容が実行された後




```
\ltjsetruby{stretchbol=121,stretcheol=121}
あ\highLight{\ruby{◆}{□□□□□□□□}}あ\hfil
い\highLight{\ruby{◆}{□□□□□□□□}}い}\\
\highLight{う\ruby{◆}{□□□□□□□□}}う\\\kern7\zw\hfil
あ\highLight{\ruby{◆|◆}{□□□□□|□□□□□}}あ

\ltjsetruby{always_highlight_protrusion}
あ\highLight{\ruby{◆}{□□□□□□□□}}あ\hfil
い\highLight{\ruby{◆}{□□□□□□□□}}い}\\
\highLight{う\ruby{◆}{□□□□□□□□}}う\\\kern7\zw\hfil
あ\highLight{\ruby{◆|◆}{□□□□□|□□□□□}}あ
```

図 2　always_highlight_protrusion キー有無による組版結果の違い

に \fontsize...\selectfont が実行されるので，このキーの指定では最後に \selectfont を加える必要はない．

この fontcmd キーの内容は多くの回数実行される．例えば，luatexja-fontspec パッケージを用いて OpenType フォントを用いる場合，

```
fontcmd=\addjfontfeatures{Style=Ruby}
```

のようにしてルビ用字形を用いることが可能だが，現在の実装ではタイプセットに時間がかかるようになる．

always_highlight_protrusion=⟨*bool*⟩　この \ltjruby 命令が lua-ul パッケージの提供する「下線」命令[*2]の引数に含まれていた場合，ルビの突出部（ただし文字進入部分は除く）に「下線」がいつでも引かれるか否か．

既定値は偽．この場合は，突出部に「下線」が引かれるのは，ルビの前後の文字も同じ「下線」命令の有効範囲になっているときに限られる．図 2 を参照．

次の 2 つは，以上で説明した複数のオプションを一度に設定するためのものである．普通はこの 2 つのうちいずれかを設定するだけで足りるだろう．

naka 以下のオプションを同時に設定する．主に中付きルビを組むときに用いる．

```
mode=1, stretch={1}{2}{1}, stretchruby={1}{2}{1}
```

kata 同様に，肩付きルビ用に，次を設定する．

```
mode=9, stretch={1}{2}{1}, stretchruby={0}{0}{1}
```

■**\ltjsetruby{**⟨*option*⟩**}**　⟨*option*⟩ の既定値を指定する．luatexja-ruby 読み込み時の値は各項目の所で既に説明してあるが，次のようになっている．

```
pre=-1, post=-1, mode=1,
stretchruby={1}{2}{1}, stretch = {1}{2}{1},
stretchbol={0}{1}{1}, stretcheol={1}{1}{0},
maxmargin=0.5, size=0.5, intergap=0, rubysmash=false,
```

[*2] \underLine, \highLight, およびそのほか．

```
kenten=\textbullet, fontcmd=\relax, ybaseheight=0.88, tbaseheight=0.5,
yrubydepth=0.12, trubydepth=0.5,
intrude_jfmgk, intrude_kanjiskip, intrude_xkanjiskip,
always_highlight_protrusion=false
```

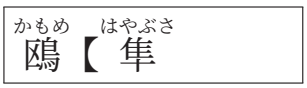
■\ltjsetparamater に追加されるキー

rubypreintrusion={⟨*chr_code*⟩, ⟨*pre_int*⟩} 文字 ⟨*chr_code*⟩ に，その**直後**のルビによって掛けられるルビ文字列の最大長（つまり，前文字進入許容量）をルビ全角幅単位で指定．

v0.5 以降での変更点：もし $-a$ $(a > 0)$ を指定した場合は，次のようになる．

- 実際の前文字進入許容量は a となる．
- ただし，文字 ⟨*chr_code*⟩ の直前のルビによって文字の前の JFM グルーに進入が発生した場合，文字 ⟨*chr_code*⟩ の直後のルビの前進入は発生しない．

後者の状況の例を以下に載せる．



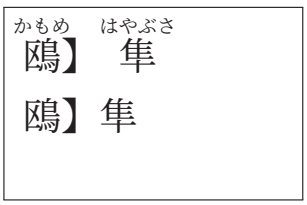

```
\ltjsetparameter{rubypreintrusion={`\【,-1}}
\ruby{鴎}{かもめ}【\ruby{隼}{はやぶさ}
% 「\ruby{鴎}{かもめ}」は，「【」直前のJFMグルーに
% ルビ半角だけ進入している
% →「\ruby{隼}{はやぶさ}」の「【」への進入はない．
```

rubypostintrusion={⟨*chr_code*⟩, ⟨*post_int*⟩} 文字 ⟨*chr_code*⟩ に，その**直前**のルビによって掛けられるルビ文字列の最大長（つまり，後文字進入許容量）をルビ全角幅単位で指定．

v0.5 以降での変更点：もし $-a$ $(a > 0)$ を指定した場合は，次のようになる．

- 実際の後文字進入許容量は a となる．
- ただし，文字 ⟨*chr_code*⟩ の直前のルビによってこの文字への実際の後文字進入が発生した場合，文字 ⟨*chr_code*⟩ の直後の JFM グルーにはルビの前進入は発生しない．

今回も後者の状況の例を以下に載せる．




```
\ltjsetparameter{rubypostintrusion={`\】,-1}}
\ruby{鴎}{かもめ}】\ruby{隼}{はやぶさ}\\
\hskip.25\zw 鴎】隼\\% 比較用
% 「\ruby{鴎}{かもめ}」は，「】」にルビ半角だけ進入
% →「\ruby{隼}{はやぶさ}」は「】」直前のJFMグルー
% には進入しない．
```

以下の文字は rubypreintrusion, rubypostintrusion とも初期値は 1 である：

ひらがな (U+3040–U+309F)，カタカナ (U+30A0–U+30FF)，仮名補助など (U+1B000–U+1B16F)，分離禁止文字 ― ‥ … 〳 〴 \ ―

次に，以下の文字は rubypreintrusion, rubypostintrusion とも初期値は 0.5 である：

中点類 • ： ；

また，以下の文字は rubypreintrusion のみ初期値は -1 である：

- 始め括弧類 ‘ “ 〈 《 「 『 【 〔 〖 〘 〚 〝 （ 〔 ｛ 〈
- 「文字コード -1 の文字[*3]」

[*3] ここでは段落開始の \parindent 分インデントを表す．通常の LuaTEX-ja における指定では「文字コード -1」は文中数式境界を表していることに注意．

改行なし（行中形）	…の表現力（ひょうげんりょく）は…
直前で改行	表現力（ひょうげんりょく）は…
途中で改行	…の表現力（ひょうげんりょく）は… …の表現力（ひょうげんりょく）は…
直後に改行	…の表現力（ひょうげんりょく）

表 1　可動グループルビの組まれ方（縦線は行の境目）

さらに，以下の文字は rubypostintrusion のみ初期値は −1 である：

- 閉じ括弧類 ’ ” 〉 》 」 』 】 〕 ） ］ ｝ 〗 〙 〛
- 句読点類 、 ， 。 ．

以上で記述されていない rubypreintrusion, rubypostintrusion の初期値はすべて 0 である．

■**\ltjkenten**　圏点を出力する命令であり，

```
\ltjkenten[⟨option⟩]{親文字}
```

のように使用する．複数文字に圏点をつける場合でも，\ltjruby のように「|」を使って文字を区切る**必要はない**（\kenten 内では「|」は特別な意味を持たない）ことに注意してほしい．

圏点として出力される文字は kenten キーによって指定し，圏点自身の大きさは（ルビと同様に）size キーで指定する．

1.3　グループの指定（してい）

```
\ltjruby[⟨option⟩]{親|文|字}{おや|も|じ}
```

のように，\ltjruby の第 2・第 3 引数内の「|」はグループの区切りを表す．グループの数は両者で一致しなければならず，\ltjruby{紋章}{もん|しよう} のようには**できない**．

1 グループのみのルビ（単純グループルビ）はグループルビとして組まれる．そのため，もしモノルビを使いたければ，面倒でも

の紋章（もんしよう）が

```
1 の\ltjruby{紋}{もん}\ltjruby{章}{しよう}が
```

のように，複数回使用すること．また，全てのグループにおいて「ルビ文字列の長さは親文字列以下」[*4]であれば，単純グループルビの並びとして扱われる．すなわち，次の 2 行は全くの等価となる．

```
\ltjruby{普|通|車}{ふ|つう|しや}
\ltjruby{普}{ふ}\ltjruby{通}{つう}\ltjruby{車}{しや}
```

複数グループかつ上記の条件を満たさない場合は，所謂「可動グループルビ」であり，ルビの前後や各グループの切れ目で行分割が可能となる．例えば

```
…の\ltjruby{表|現|力}{ひよう|げん|りよく}は…
```

という入力からは得られる組版結果は，表 1 のいずれかになる．

[*4] 実際には TEX での長さの計算誤差（2^{-16} pt の整数倍として計算していることによる）を考慮し，親文字全角の 1/1000 だけルビ文字列が長くなることを許容している．

- これらの行分割によってペナルティは発生しない．
- 上記の例で見られるように，2 ブロック以上をまとめて組むときは，全体を 1 つのグループルビのように組版する（JIS X 4051 と同様）．『日本語組版処理の要件』では，附属書 F に「熟語の構成，さらにその熟語の前後にくる文字の種類を考慮して配置する方法」として別の方法を解説しているが，こちらの方法は現時点ではサポートしていないので，面倒でも手動で pre, post などを使って頑張って欲しい．
- 実装方法の都合により，ルビの直前・直後・途中で 2 箇所以上の改行が起きる場合，例えば

  …の表現力　　表現力は…　　表現力

  などの組み方は**想定していない**．エラーが発生して止まることもあるし，エラーが発生しなくても正しく組まれない．

■別名について　\begin{document}時に

- \ruby がすでに定義されていない場合に限り，\ruby が \ltjruby の別名として定義される．
- \kenten がすでに定義されていない場合に限り，\kenten が \ltjkenten の別名として定義される．

2　注意点

■前後からのルビ文字のはみ出し 1　「日本語組版処理の要件」の図 117[*5]のように，前後からのルビ文字のはみ出しが繋がらないようにする処理が組み込まれている．例えば，

陵と陵
陵と陵

```
1 \ltjruby{陵}{りょう}と\ltjruby{陵}{みささぎ}\\
2 \ltjruby{陵}{りょう}と\ltjruby[pre=1]{陵}{みささぎ}
```

において，1 行目右側の「陵」のルビが前の「と」にかかる量は次のように決まる：

1. 1 回目の実行では，行分割前に「陵」の後文字進入量は前もって知ることはできない．そのため，「陵」は行中形で組まれるものと想定し，「陵」の前文字進入許容量は

$$\underbrace{0.5\,\mathrm{zw}}_{\text{元々の許容量}} - \underbrace{0.25\,\mathrm{zw}}_{\text{前のルビの後文字進入量（行中形）}} = 0.25\,\mathrm{zw}$$

   となる．行分割後，「陵」の実際の後文字進入量は ⟨*jobname*⟩.ltjruby ファイルに記述される．
2. 2 回目以降の実行では，⟨*jobname*⟩.ltjruby ファイルに保存された「陵」の後文字進入量を用いて，「陵」の前文字進入許容量を次のように計算する：

$$\underbrace{0.5\,\mathrm{zw}}_{\text{元々の許容量}} - \underbrace{0.25\,\mathrm{zw}}_{\text{前のルビの後文字進入量 (from .ltjruby)}} = 0.25\,\mathrm{zw}.$$

⟨*jobname*⟩.ltjruby ファイルに保存する際，各 \ltjruby 命令の呼び出しを識別するキーが必要になるが，そのキーとしては単純に「何個目の \ltjruby 命令か」である．

なお，以上の処理は，1 行目と 2 行目を比較すれば分かるように，「陵」の**前文字進入許容量指定が自動（pre の値が負）**になっている場合のみ実施される．

[*5] 2020-08-11 版での番号．2012-04-03 版では図 3.82.

■前後からのルビ文字のはみ出し 2　また，本パッケージでは，「日本語組版処理の要件」にある

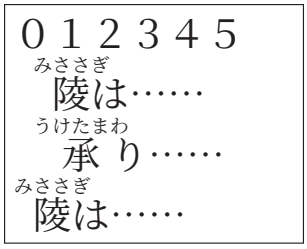
> 後ろにくる終わり括弧類，句点類若しくは読点類，又は前にくる始め括弧類には，最大でルビ文字サイズの全角までルビ文字を掛けてもよい．この場合，後ろにくる終わり括弧類，句点類若しくは読点類の後ろの空き量，又は前にくる始め括弧類の前の空き量に掛けてはならない．

という処理も組み込まれており，⟨*jobname*⟩`.ltjruby` に「前後の和文処理グルーに正の量だけ進入したか」という情報を保存することによって実装されている．

■段落末尾のルビ　段落がルビで終わった場合，そのルビが行末形で組まれることはない．これは，段落の「本当の」末尾には `\penalty10000\parfillskip` があるためで，ルビ処理用に作った最後のグルー（3.2 小節の説明では g_2）が消去されないことによる．

`\parfillskip`（や，場合によっては `\rightskip`）の長さを実測し，それによって処理を変えるのも可能だが，そのようなことはしなかった．段落がルビで終わることは普通ない（最低でも句点が続くだろう）と考えられるからである．

■段落先頭のルビ　同様に，段落先頭のルビは行頭形にはならない．`pre` が負（つまり，自動指定）のとき，段落最初の `\parindent` 分への進入は可能である．この `\parindent` 分のインデントへの進入許容量は「文字 −1」に対する rubypreintrusion（既定値は 1，ルビ全角単位）と `\parindent` の長さのうち小さい方である．




```
\parindent1\zw\noindent ０１２３４５\par
\ltjruby{陵}{みささぎ}は……\par
\ltjruby{承}{うけたまわ}り……\par
\parindent0.25\zw\ltjruby{陵}{みささぎ}は……
```

■和文処理グルーの伸縮　現バージョンでは，進入量調整に和文処理グルーを考慮させる，

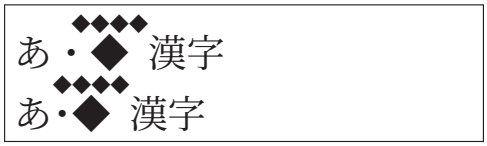
> `intrude_jfmgk`, `intrude_kanjiskip`, `intrude_xkanjiskip` キー

の値が `true`（真）であった状況でも，考慮されるのは**自然長の値のみ**である．そのため，行の調整処理が発生した場合は意図しない結果となる．

例えば，標準設定での中黒「・」の直後のルビからの中黒への進入許容量は

> 中黒「・」への前進入許容量はルビ全角の半分で，中黒本体の後の四分空きには進入可能

となっている．そのため，下の例の 1, 2 行目ともルビの前進入量は 0.5 zw となる．しかし，2 行目では詰め量の 0.5 zw がほとんど中黒周囲の四分空きで負担されるため，実際には「中黒本体にほぼ 0.5 zw が進入する」という望ましくない結果が得られている．




```
\leavevmode\hbox{あ・\ltjruby{◆}{◆◆◆◆}漢字}\\
\hbox spread-0.5\zw{あ・\ltjruby{◆}{◆◆◆◆}漢字}
```

また，次の例では 2 行目では前後の文字進入量がルビ 1 字分と等しくなっているが，3 行目のように 1 全角伸ばすという調整が行われた後は文字進入量が不揃いになってしまっている．これはもともと「;」には後側にのみ和欧文間空白が入ることと，3 行目ではこの和欧文間空白が伸びているためである．

セミコロン
に ; という
セミコロン
に ; と い う

```
1 \leavevmode\hbox{に\ltjruby{\texttt{;}}{セミコロン}という}\\
2 \hbox spread\zw{に\ltjruby{\texttt{;}}{セミコロン}という}
```

3 実装(じっそう)について

3.1 進入量の計算

ルビ文字を自然に組んだときの幅が親文字のそれより多い場合，ルビの前後への進入量は次のように決定される．

Step 1 前文字進入許容量 B_0，後進入文字許容量 A_0 の算出．

ルビ全角の長さを r とする．

(a) `pre` の指定値が非負であった場合は，それに r を掛けたものを B_0 とする．
そうでなかった場合は，「ルビの直前の文字」に対する rubypreintrusion の絶対値に r を掛けたものを B_0 とする[*6]．ここで，もし 2 つ前の文字がルビで，その直後（つまりいま処理しているルビから見れば直前）の文字へ a' だけの進入があった場合，現在のルビについて $B_0 \leftarrow \min(0, B_0 - a')$ とする．

(b) `post` の指定値が非負であった場合は，それに r を掛けたものを A_0 とする．
そうでなかった場合は，「ルビの直後の文字」に対する rubypostintrusion の絶対値にルビ全角の値を掛けたものを A_0 とする．

(c) `mode` の最下位ビット (bit 0) が 0 であった場合は，$B_0 \leftarrow 0$, $A_0 \leftarrow 0$ とする．

Step 2 前進入許容量 B，後進入許容量 A の算出．

(a) まず $B \leftarrow B_0$，$A \leftarrow A_0$ とする．

(b) ルビとその直前の文字の間に和文処理グルー g が挿入された場合，

- g が JFM グルーの場合は，`intrude_jfmgk` が真の場合に，
- g が標準の和文間空白 (kanjiskip) の場合は，`intrude_kanjiskip` が真の場合に，
- g が標準の和欧文間空白 (xkanjiskip) の場合は，`intrude_xkanjiskip` が真の場合に，

それぞれ g の自然長を B に加算する．
ルビとその直後の文字との間に和文処理グルーが挿入された場合も同様である．

(c) `mode` の 2 番目のビット (bit 1) が 1 であった場合は，B, $A \leftarrow \min(B, A)$ とする．

Step 3 実際の前進入量 b，後進入量 a の計算．

ルビ文字の突出量を x，親文字の文字数を $k+1$，親文字の前に入る空白量・間の空白量・後ろの空白量の比を $p : q : r$ とする．このとき，`mode` の bit 2–4 の値によって b, a を次のように算出する：

000 $b = \min(B, xp/(p+kq+r))$，$a = \min(A, xr/(p+kq+r))$

001 $b = \min(B, x)$，$a = \min(A, \max(x-b, 0))$

010 $a = \min(A, x)$，$b = \min(B, \max(x-a, 0))$

011 $M = \min(B, A)$ とおく．もし $x \le 2M$ ならば $b = a = x/2$．そうでなければ

$$b = \min\left(B, M + \frac{(x-2M)p}{p+kq+r}\right), \quad a = \min\left(A, M + \frac{(x-2M)r}{p+kq+r}\right).$$

100 $M = \min(B, A)$ とおく．もし $x \le 2M$ ならば $b = a = x/2$．そうでなければ

$$b = \min(B, x-M), \quad a = \min(A, x-M) \quad \text{（どちらかは必ず } M \text{ となる）}.$$

[*6] なお，「ルビの直前の文字」が段落最初の `\parindent` 分のインデントであった場合，B_0 を，「文字コード -1 の文字」rubypreintrusion の値に r を掛けた値と `\parindent` のうち小さい方とする．

大雑把に言うと，最初の「000」は「組んだ後に進入量を決定」する方針，それ以外は「進入量を先に決定し，その後で組む」方針である．組み方の具体例を実際に示す．例示のため，平仮名にはルビが 1 字まで，「立」にはルビを 0.5 字分までかけてよいことにしている．

000 は美(うつく)しい　は聴(ちようしゆう) 衆と　は暁(あかつき) 立　は峠(とうげ)立　は聴(ちようしゆう) 衆立　は聴(ちようしゆう) 衆 達
001 は美(うつく)しい　は聴衆(ちようしゆう)と　は暁(あかつき)立　は峠(とうげ)立　は聴衆(ちようしゆう)立　は聴(ちようしゆう)衆達
010 は美(うつく)しい　は聴衆(ちようしゆう)と　は暁(あかつき)立　は峠(とうげ)立　は聴衆(ちようしゆう)立　は聴(ちようしゆう)衆達
011 は美(うつく)しい　は聴衆(ちようしゆう)と　は暁(あかつき)立　は峠(とうげ)立　は聴衆(ちようしゆう)立　は聴(ちようしゆう) 衆 達
100 は美(うつく)しい　は聴衆(ちようしゆう)と　は暁(あかつき)立　は峠(とうげ)立　は聴衆(ちようしゆう)立　は聴(ちようしゆう)衆達

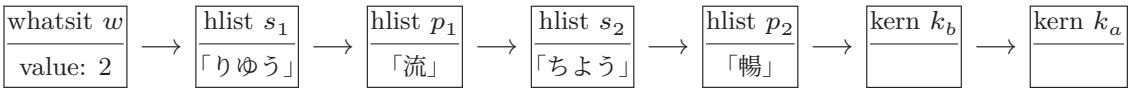
3.2　ノードの扱い

次の例で内部実装の大まかな方法を説明する．

……を流(りゆうちよう) 暢に……

```
1 ……を\ltjruby{流|暢}{りゆう|ちよう}に……
```

1. `\ltjruby` 命令自体は，一旦次の node list を値とする whatsit W を作って，現在の水平リストへと挿入する（必要ならば `\leavevmode` も実行）：



ここで，最初の w の値 2 は，ルビが 2 つのパーツ「流(りゆう)」「暢(ちよう)」からなっていることを表している．この値を cmp とおこう．s_i 達の中の文字は既にルビの大きさである．
また，k_b, k_a は `always_highlight_protrusion` キーが偽の場合にのみ生成されるが，実際に使われるのは行分割後になってからである．lua-ul 対応のためには，和文処理グルーの挿入処理で「前・後のクラスタそれぞれとの attribute の共通部分」を保存する必要があり，この段階でノードとして k_b, k_a を準備している．

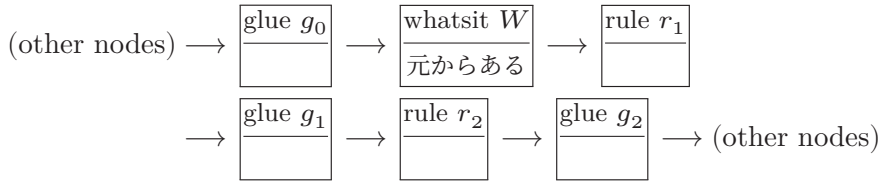
2. LuaTEX-ja の和文処理グルー挿入処理において，この whatsit W はまとめて「先頭が『流』，最後が『暢』であるような hbox を `\unhbox` で展開したもの」と扱われる．言い換えれば，ルビ部分を無視した単なる「流暢」という和文文字の並びとして扱われる[*7]．次のサンプルを参照

. A
. A

```
1 \leavevmode\hbox{. }A\\
2 %↑xkanjiskip
3 \ltjruby{. }{}A
4 %↑2分
```

3. 和文処理グルーの挿入が終わった後で，可動グループルビのためのノードの挿入に入る．

（a）W の前後に $2cmp+1=5$ 個のノードが挿入され，W の周辺は次のようなノード列になる[*8]．



（b）このようにノードを挿入する目的は，TEX の行分割処理自体に影響を加えずに可動グループ

[*7] 「流」「暢」の間のグルーは既に入っている，と扱われる．
[*8] r_2 と g_2 の間には，ルビと直後の文字の間にある禁則処理のためのペナルティなどが入っているかもしれない．

ノード	組み方	サンプル	対応するノード並び
n_1	行末 1 グループ	を流(りゅう)	$g_0 \to W \to r_1$
n_2	行末 2 グループ	を流(りゅう) 暢(ちょう)	$g_0 \to W \to r_1 \to g_2 \to r_2$
n_3	行頭 2 グループ	流(りゅう) 暢(ちょう)に	$W \to r_1 \to g_2 \to r_2 \to g_2$
n_4	行頭 1 グループ	暢(ちょう)に	$r_2 \to g_2$
n_5	行中	を流(りゅう) 暢(ちょう)に	$g_0 \to W \to r_1 \to g_2 \to r_2 \to g_2$

表 2　ルビの組み方と対応するノード並び

ルビを実現させることにある．

$$\text{(other nodes)} \longrightarrow \boxed{\text{glue } g_0} \longrightarrow \boxed{\begin{array}{c}\text{whatsit } W\\ \hline \text{元からある}\end{array}} \longrightarrow \boxed{\text{rule } r_1}$$
$$\text{行の境目}$$
$$\boxed{\text{rule } r_2} \longrightarrow \boxed{\text{glue } g_2} \longrightarrow \text{(other nodes)}$$

のようになったとしたら，「流(りゅう)」「暢(ちょう)」の間で行分割が起きた，ということがわかり，g_i, r_i 達のノードを適切に置き換えればよい（後で詳しく説明する）．

(c) なお，r_i 達の高さ・深さは組み上がった後のそれである．g_i, r_i 達の幅は，表 2 に示したような対応に沿って算出する．例えば，行中形 n_5 に対して，記号は乱用気味であるが，

$$g_0 + r_1 + g_2 + r_2 + g_2 = (3 - 0.25 \times 2)\,\mathrm{zw} = 2.5\,\mathrm{zw}$$

という方程式が立つ（zw は親文字全角の幅，引き算は進入量の分）．方程式と未知数（g_i, r_i 達の幅）はどちらも $2cmp + 1$ であるので，Gauß の消去法により g_i, r_i 達の幅が求まる．

(d) また，ルビ処理を統括している whatsit W の値も

$$\boxed{\begin{array}{c}\text{whatsit } w\\ \hline \text{value: } 2\end{array}} \longrightarrow \boxed{\begin{array}{c}\text{vlist } n_1\\ \hline \text{末 1}\end{array}} \longrightarrow \boxed{\begin{array}{c}\text{vlist } n_2\\ \hline \text{末 2}\end{array}} \longrightarrow \boxed{\begin{array}{c}\text{vlist } n_3\\ \hline \text{頭 2}\end{array}} \longrightarrow \boxed{\begin{array}{c}\text{vlist } n_4\\ \hline \text{頭 1}\end{array}} \longrightarrow \boxed{\begin{array}{c}\text{vlist } n_5\\ \hline \text{中}\end{array}}$$
$$\longrightarrow \boxed{\text{kern } k_b} \longrightarrow \boxed{\text{kern } k_a}$$

に置き換えておく．

4. LuaTEX の行分割処理を通常通りに行う．
5. 行分割の結果に従って，g_i, r_i 達を適切に置換する．
   例えば行分割の結果

$$\text{(other nodes)} \longrightarrow \boxed{\text{glue } g_0} \longrightarrow \boxed{\begin{array}{c}\text{whatsit } W\\ \hline \text{元からある}\end{array}} \longrightarrow \boxed{\text{rule } r_1} \qquad \text{(行 A)}$$
$$\text{行の境目}$$
$$\boxed{\text{rule } r_2} \longrightarrow \boxed{\text{glue } g_2} \longrightarrow \text{(other nodes)} \qquad \text{(行 B)}$$

のようになったとしよう．

(a) 処理は段落の上の行から順番に行われる．行 A の処理がまわってきたとしよう．

(b) 行 A の先頭から順番に眺めていく．すると「whatsit W 由来」のノード，g_0, W, r_1 が見つかり，行 A はここで終わっている．

まず，行 A の hbox の中身から whatsit W を消去（リストから取り除くだけで，W のメモリを解放するわけではない）する．g_0, (W,) r_1 というノードの並びは，「行末 1 グループ」n_1 に対応しているので，g_0, r_1 を行 A から除去・メモリ解放し，代わりに n_1 を行 A の中身に追加する．

(c) `always_highlight_protrusion` キーが偽であり，かつルビの前への突出量から前文字進入量を引いた値 $prot_b$ が 0 でない場合は，n_1 の直前に kern k_b（長さ $prot_b$）と，それを打ち消す kern（長さ $-prot_b$）を挿入する．

(d) 次に行 B の処理にうつる．行 A でルビの処理は完了していない（2 グループのルビなのにまだ 1 グループしか使っていない）ので，「whatsit W 由来」のノードがいくつか行 B 内に残っているはずである．

案の定，r_2, g_2 というノード列が見つかった．これは「行頭 1 グループ」n_4 に対応しているので，r_2, g_2 を行 B から除去・メモリ解放し，代わりに n_4 を行 B の中身に挿入する．`always_highlight_protrusion` キーが偽であったときは，行 A のときと同様に kern を 2 つ挿入する．

(e) これで 2 グループとも使い切ったことになるので，隔離しておいた W を，（使われなかった n_2, n_4, n_5 などと共に）メモリ解放する．結果として次のようになる．

(other nodes) ⟶ kern k_b ⟶ kern $-prot_b$ ⟶ vlist n_1 末 1 (行 A)

行の境目

vlist n_4 頭 1 ⟶ kern $-prot_a$ ⟶ kern k_a ⟶ (other nodes) (行 B)

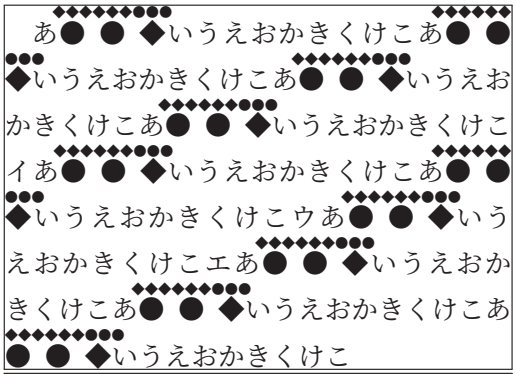
ここで，色付きの 4 ノードは，lua-ul パッケージの提供する「下線」命令が `\ltjruby` のみを有効範囲として発行された場合の，「下線」がかかってくるノードたちである．

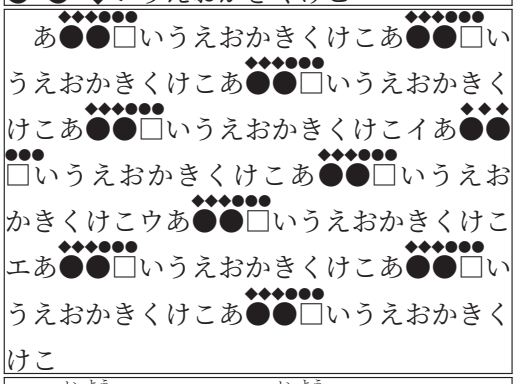
4 いくつかの例

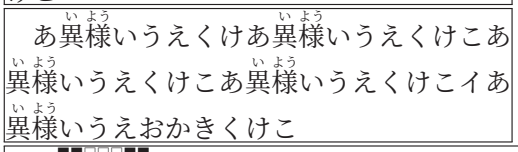
ああああ黄金橋いうえおかきくけこあ
黄金橋いうえおかきくけこあ黄金橋
いうえおかきくけこあ黄金橋いうえおか
きくけこあ黄金橋いうえおかきくけこあ
黄金橋いうえおかきくけこ

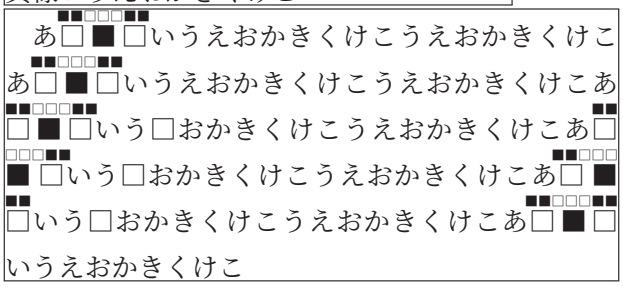
`baseheight=0.88` での例
ふぁいる dvi ファイル oo 漢字
ファイル dvi ふぁいる oo 漢字
ああ（ほげ）」ほげ「【ほげ】ほげ…

あ国府津いうえおかきくけこあ国府津
いうえおかきくけこあ国府津いうえおか
きくけこあ国府津いうえおかきくけこあ
国府津いうえおかきくけこあ国府津いう
えおかきくけこ





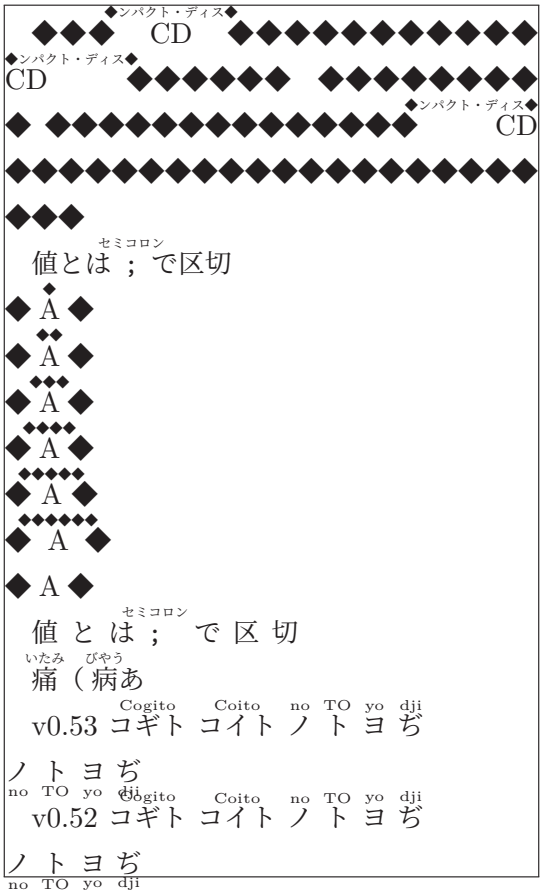




標準 又承る 疎 は俄勉強 後俄勉強 は暁には俄に 休憩中かつ視聴中
肩つき 又承る 疎 は俄勉強 後俄勉強 は暁には俄に 休憩中かつ視聴中

休憩中かつ視聴中 **勉強と勉強** コギト・エルゴ・スム Cogito ergo sum

```
{\ltjsetruby{stretch=101}% 親文字均等割り禁止
    \ruby{休|憩|中}{きゆう|けい|ちゆう}かつ\ruby{視|聴|中}{し|ちよう|ちゆう}%
}\quad
\textgt{\ruby{勉|強}{べん|きよう}と%
    \ruby[fontcmd=\mcfamily]{勉|強}{べん|きよう}}\quad% ルビは明朝体
\ruby{コギト・エルゴ・スム}{Cogito ergo sum}\quad    % 欧文空白は伸長しない
\ruby[size=1]{Cogito ergo sum}{コギト・エルゴ・スム}% 欧文空白は伸長しない
```



たとえば取消線を引いてみる．取消線だとどうかな

周囲が aaaaaaaaaaa ない あああああああああああああああああああああああああああああ周囲が あ い う え ないと思う

```
% lua-ul
たとえば\strikeThrough{\ruby{取|消|線}{とりけしの|いちじゆう|せん}を}引いてみる．
\strikeThrough{\ruby{取|消|線}{とりけしの|いちじゆう|せん}}だとどうかな

\ltjsetparameter{prebreakpenalty={`な,2893}}% デバッグ用．あっても正常動作を確認．
\highLight{周囲が}\ruby{aaaaaaaaaaa}{全く}\highLight{ない}
あああああああああああああああああああああああああああああ
周囲が\highLight{\ruby{あ|い|う|え}{おそらく|きっと|たぶん|maybe}}ないと思う
```

5　『日本語組版処理の要件』20120403 の例

■3.3.1 節

3.49 君子(くんし)は和(わ)して同(どう)ぜず
3.50 人(ひと)に誨(おし)えて倦(う)まず
3.51 鬼門(きもん)の方角(ほうがく)を凝視(ぎょうし)する
3.52 鬼門(きもん)の方角(ほうがく)を凝視(ぎょうし)する
3.53 茅場町(かやばちょう)　茅場町(かやばちょう)
3.54 紫陽花(あじさい)　坩堝(るつぼ)　田舎(いなか)
3.55 模型(モデル)　顧客(クライアント)　境界面(インターフェース)　避難所(アジール)
3.56 編集者(editor)　editor(エディター)

```
\obeylines
3.49 \ruby{君|子}{くん|し}は\ruby{和}{わ}して\ruby{同}{どう}ぜず
3.50 \ruby{人}{ひと}に\ruby{誨}{おし}えて\ruby{\倦}{う}まず
% モノルビ．面倒でも複数回の実行が必要
3.51 \ruby{鬼}{き}\ruby{門}{もん}の\ruby{方}{ほう}\ruby{角}{がく}を%
    \ruby{凝}{ぎょう}\ruby{視}{し}する
% 熟語ルビ
3.52 \ruby{鬼|門}{き|もん}の\ruby{方|角}{ほう|がく}を\ruby{凝|視}{ぎょう|し}する
3.53 \ruby{茅場町}{かやばちょう}\quad\ruby{茅|場}{かや|ば}\ruby{町}{ちょう}
% 熟字訓
3.54 \ruby{紫陽花}{あじさい}\quad\ruby{坩堝}{るつぼ}\quad\ruby{田舎}{いなか}
% グループルビ
3.55 \ruby{模型}{モデル}\quad\ruby{顧客}{クライアント}\quad%
    \ruby{境界面}{インターフェース}\quad\ruby{避難所}{アジール}
3.56 \ruby{編集者}{editor}\quad \ruby{editor}{エデイター}
```

■3.3.3 節

3.58 に幟(のぼり)を　に幟(のぼり)を　韋編三絶(いへんさんぜつ)
3.59 に幟(のぼり)を
3.60 韋編三絶(いへんさんぜつ)　韋編三絶(いへんさんぜつ)

```
\obeylines
3.58 に\ruby{幟}{のぼり}を\quad に\ruby[kata]{幟}{のぼり}を\quad%
    \ruby{韋}{い}\ruby{編}{へん}\ruby{三}{さん}\ruby{絶}{ぜつ}
% 三分ルビ．JY3/mc/mc は本文書のプリアンブルで独自に定義
3.59 に\ruby[fontcmd=\kanjifamily{mc}\kanjiseries{mc}]{幟}{のぼり}を
% ルビ文字を小さくする
3.60 {\Large%
  \ruby{韋}{い}\ruby{編}{へん}\ruby{三}{さん}\ruby{絶}{ぜつ}\quad% 比較用
  \ltjsetruby{size=0.375}% 0.5 -> 0.375
  \ruby{韋}{い}\ruby{編}{へん}\ruby{三}{さん}\ruby{絶}{ぜつ}}
```

■3.3.4 節　3.61 図（両側ルビ）は未サポートにより省略

■3.3.5 節 モノルビ

3.62 の薬(やく)に
3.63 版面の地(ち)に　版面の地(ち)に
3.65 の砦(とりで)に
3.66 上 の旬(しゆん)に　後旬(しゆん)に
3.66 下 の旬(しゆん)又　後旬(しゆん)又

```
\obeylines
3.62 の\ruby{薬}{やく}に
3.63 版面の\ruby{地}{ち}に\quad 版面の\ruby[kata]{地}{ち}に
3.65 の\ruby{砦}{とりで}に
{\ltjsetruby{kata}%
3.66上 の\ruby{旬}{しゆん}に\quad 後\ruby{旬}{しゆん}に
3.66下 の\ruby{旬}{しゆん}又\quad 後\ruby{旬}{しゆん}又}
```

■3.3.6 節 グループルビ

3.67 は冊子体(コーデツクス)と
3.68 模型(モデル)　利用許諾(ライセンス)
3.69 模型(モデル)　利用許諾(ライセンス)
3.70 なげきの聖母像(ピエタ)　なげきの聖母像(ピエタ)
3.71 顧客(クライアント)　境界面(インターフエース)
3.72 顧　客(クライアント)　境　界　面(インターフエース)

```
\obeylines
3.67 は\ruby{冊子体}{コーデツクス}と
3.68 \ruby{模型}{モデル}\quad \ruby{利用許諾}{ライセンス}
% 両端を揃える流儀
3.69 {\ltjsetruby{stretchruby=010}%
    \ruby{模型}{モデル}\quad \ruby{利用許諾}{ライセンス}}
% ルビが極端に短い場合
3.70 \ruby{なげきの聖母像}{ピエタ}\quad% ルビ全角まで許容
    \ruby[maxmargin=0.75]{なげきの聖母像}{ピエタ}\\% ルビ全角1.5倍まで
% ルビが長い場合
3.71 \ruby{顧客}{クライアント}\quad \ruby{境界面}{インターフエース}
3.72 {\ltjsetruby{stretch=010, stretchbol=010, stretcheol=010}% はみ出さない流儀
    \ruby{顧客}{クライアント}\quad \ruby{境界面}{インターフエース}}
```

■3.3.7 節 熟語ルビ

3.73 杞憂　畏怖　杞憂　畏怖
3.74 の流儀を　の無常を　の成就を　の紋章を　の象徴を
3.75 の流儀を　の無常を　の成就を　の紋章を　の象徴を
3.76× の流儀を　の無常を
3.77 ……等を流儀が　……等の無常を

```
\obeylines
3.73 \ruby{杞|憂}{き|ゆう}\quad \ruby{畏|怖}{い|ふ}\quad%
    \ruby[kata]{杞|憂}{き|ゆう}\quad \ruby[kata]{畏|怖}{い|ふ}
3.74 の\ruby{流|儀}{りゅう|ぎ}を\quad　の\ruby{無|常}{む|じょう}を\quad%
    の\ruby{成|就}{じょう|じゅ}を\quad の\ruby{紋|章}{もん|しょう}を\quad%
    の\ruby{象|徴}{しょう|ちょう}を
% 熟語の構成を考慮した方法は現行ではできない．
% 行分割はできるのだが，「他の漢字にルビ全角までかかって良い」は難しい．
3.75 {\ltjsetruby{kata}%
    の\ruby{流|儀}{りゅう|ぎ}を\quad　の\ruby{無|常}{む|じょう}を\quad%
    の\ruby{成|就}{じょう|じゅ}を\quad の\ruby{紋|章}{もん|しょう}を\quad%
    の\ruby{象|徴}{しょう|ちょう}を}
% モノルビ配置．望ましくない
3.76× の\ruby{流}{りゅう}\ruby{儀}{ぎ}を\quad の\ruby{無}{む}\ruby{常}{じょう}を
3.77 {\ltjsetruby{stretchbol=121, stretcheol=121}% 行頭・行末揃えず
    \hbox{\vrule\vbox{\hsize=5.25\zw ……等を\ruby{流|儀}{りゅう|ぎ}が}\vrule}\quad%
    \hbox{\vrule\vbox{\hsize=5\zw ……等の\ruby{無|常}{む|じょう}を}\vrule}}
```

■3.3.8 節 ルビはみ出し

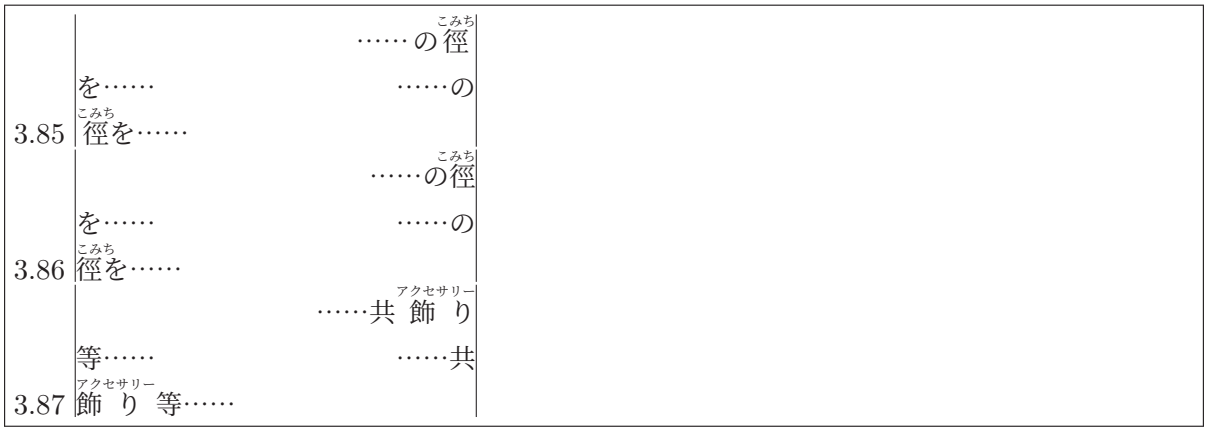
3.78 人は死して名を残す
3.79 漢字の部首には偏・冠・脚・旁がある
3.79 漢字の部首には偏，冠，脚，旁がある
3.79 漢字の部首には偏，冠，脚，旁がある

```
\obeylines\RULER{3.78}{4}
3.78 \ruby{人}{ひと}は\ruby{死}{し}して\ruby{名}{な}を\ruby{残}{のこ}す
3.79 漢字の部首には\ruby{偏}{へん}・\ruby{冠}{かんむり}・\ruby{脚}{きやく}・%
    \ruby{旁}{つくり}がある
3.79 {\ltjsetparameter{rubypreintrusion={`\，,1}}%
漢字の部首には\ruby{偏}{へん}，\ruby{冠}{かんむり}，\ruby{脚}{きやく}，%
\ruby{旁}{つくり}がある}
% 上が『要件』にある例．標準（下）では句読点の直前にはルビがかからない．
3.79 漢字の部首には\ruby{偏}{へん}，\ruby{冠}{かんむり}，\ruby{脚}{きやく}，%
\ruby{旁}{つくり}がある
```

3.79 この噂(うわさ)の好きな人は懐(ふところ)ぐあいもよく、檜(ひのき)を
3.80 漢字の部首には「偏(へん)」「冠(かんむり)」「脚(きゃく)」「旁(つくり)」がある
3.80 この噂(うわさ)好きな人は懐(ふところ)具合もよく、檜(ひのき)材を
3.81× に暁(あかつき)の趣(おもむき)を（良くない例）
3.82 に暁(あかつき)の 趣(おもむき)を
3.83 この噂(うわさ)の好きな人は懐(ふところ)ぐあいもよく、檜(ひのき)を
3.83 この噂(うわさ)好きな人は懐(ふところ)具合もよく、檜(ひのき)材を
3.84 この 噂(うわさ) の好きな人は 懐(ふところ) ぐあいもよく、檜(ひのき) を
3.84 この 噂(うわさ) 好きな人は 懐(ふところ) 具合もよく、檜(ひのき) 材を

```
\obeylines\RULER{3.79}{9}
3.79 この\ruby{\噂}{うわさ}の好きな人は\ruby{懐}{ふところ}ぐあいもよく、\ruby{檜}{ひのき}を
3.80 漢字の部首には「\ruby{偏}{へん}」「\ruby{冠}{かんむり}」「\ruby{脚}{きやく}」%
「\ruby{旁}{つくり}」がある
3.80 {\ltjsetparameter{rubypreintrusion={`\, ,1}}%
この\ruby{\噂}{うわさ}好きな人は\ruby{懐}{ふところ}具合もよく、\ruby{檜}{ひのき}材を}
3.81× に\ruby{暁}{あかつき}の\kern-1\zw の\ruby{趣}{おもむき}を（良くない例）
3.82 に\ruby{暁}{あかつき}の\ruby{趣}{おもむき}を

% 漢字・ひらがな・カタカナにルビを2分まで掛けても良い流儀
{%
  \makeatletter\count@="3040\loop\relax\ifnum \count@<"30FF%
  \ltjsetparameter{rubypreintrusion={\the\count@,0.5}, %
    rubypostintrusion={\the\count@,0.5}}%
  \advance\count@1 \repeat
  \ltjsetparameter{rubypostintrusion={`好,0.5}}
  \ltjsetparameter{rubypostintrusion={`具,0.5}}
  \ltjsetparameter{rubypostintrusion={`材,0.5}}
3.83 この\ruby{\噂}{うわさ}の好きな人は\ruby{懐}{ふところ}ぐあいもよく、\ruby{檜}{ひのき}を
3.83 この\ruby{\噂}{うわさ}好きな人は\ruby{懐}{ふところ}具合もよく、\ruby{檜}{ひのき}材を
}
% 平仮名にもルビを掛けない流儀
{%
  \makeatletter\count@="3040\loop\relax\ifnum \count@<"30A0%
  \ltjsetparameter{rubypreintrusion={\the\count@,0}, %
    rubypostintrusion={\the\count@,0}}%
  \advance\count@1 \repeat
3.84 この\ruby{\噂}{うわさ}の好きな人は\ruby{懐}{ふところ}ぐあいもよく、\ruby{檜}{ひのき}を
3.84 この\ruby{\噂}{うわさ}好きな人は\ruby{懐}{ふところ}具合もよく、\ruby{檜}{ひのき}材を
}
```




```
\obeylines
3.85\ {\ltjsetruby{stretchbol=121, stretcheol=121}% 行頭・行末揃えず
\hbox{\vrule\vbox{\hsize=15\zw
  \hskip9.5\zw……の\ruby{径}{こみち}を……%
\hskip9\zw……の\ruby{径}{こみち}を……}\vrule}}
3.86\ \hbox{\vrule\vbox{\hsize=15\zw
  \hskip10\zw……の\ruby{径}{こみち}を……%
\hskip9\zw……の\ruby{径}{こみち}を……}\vrule}
3.87\  \hbox{\vrule\vbox{\hsize=15\zw
  \hskip8\zw……共\ruby{飾り}{アクセサリー}等……%
\hskip9\zw……共\ruby{飾り}{アクセサリー}等……}\vrule}
```

■圏点の例（常用漢字表前書きより）　この表は，法令，公用文書，新聞，雑誌，放送など，一般の社会生活において現代の国語を書き表す場合の漢字使用の目安を示すものである。

「﹅」の大きさを親文字の 0.33 倍にした例　この表は，法令，公用文書，新聞，雑誌，放送など，一般の社会生活において現代の国語を書き表す場合の漢字使用の目安を示すものである。